

Worksheet 1 — Security Mindset & Threat Modeling (3 hrs)

Course: Software Security (KOSEN69) · **Week 1**

Aligned to: OWASP 2025 A06 Insecure Design · CWE-501 (Trust Boundary Violation)

Signature game: "Elevation of Privilege" (Microsoft STRIDE card deck)

Ethics note: This week is *modeling only* — you analyze design, you do **not** attack the app. Run the sample app only on your own VM/localhost. Never apply these techniques to systems you do not own or lack written permission to test.

Part 1 — Student Information

Name	Student ID	Date	Group
Sutapat Chucham	6531503085	30 Aug 2026	

Part 2 — Lecture Questions

Answer in your own words (2–4 sentences each).

1. Define the CIA triad and give one concrete failure example for each of the three properties.

Ans:

Trust boundary is a solutions that define, enforce, or visualize zero-trust perimeters across devices, applications, and infrastructure. And without adequate protective measures, data could be leaked, hacked, or misused.

2. What is a *trust boundary*, and why does data crossing one deserve extra scrutiny?

Ans:

Trust boundary is a solutions that define, enforce, or visualize zero-trust perimeters across devices, applications, and infrastructure. And without adequate protective measures, data could be leaked, hacked, or misused.

3. Explain "attack surface." Name two things that increase it in a web app.

Ans:

An attack surface is the entire area of an organization or system that is susceptible to hacking.

1. Authentication & Input Points like Exposing Unrestricted User Input & File Uploads and Complex or Multiple Authentication Paths. 2. Data & Storage Exposure like Exposing Sensitive Data via URL Parameters

4. What does each STRIDE letter map to, and which security property does each threat violate?

Ans:

Spoofing → violates authentication → pretending to be someone you're not. Tampering → violates integrity → modifying data or code you shouldn't.

Repudiation → violates non-repudiation → doing something and being able to deny it.

Information disclosure → violates confidentiality → leaking data.

Denial of service → violates availability → making it unusable. Elevation of privilege → violates authorization → gaining rights you shouldn't have

5. What does "Secure by Design" (CISA) mean, and how does it differ from bolting security on after release?

Ans:

means security is a design property decided up front, not a patch bolted on after release. While bolting on fixes specific cases and leaves the next comparable error unprotected, Secure by Design prevents entire categories of dangers. While patching is reactive and iterative, design is proactive and scalable.

Part 3 — Hands-on Lab (180 min)

Learning goals: build a data-flow diagram (DFD), apply STRIDE to a real Flask app, rank risks, and propose mitigations.

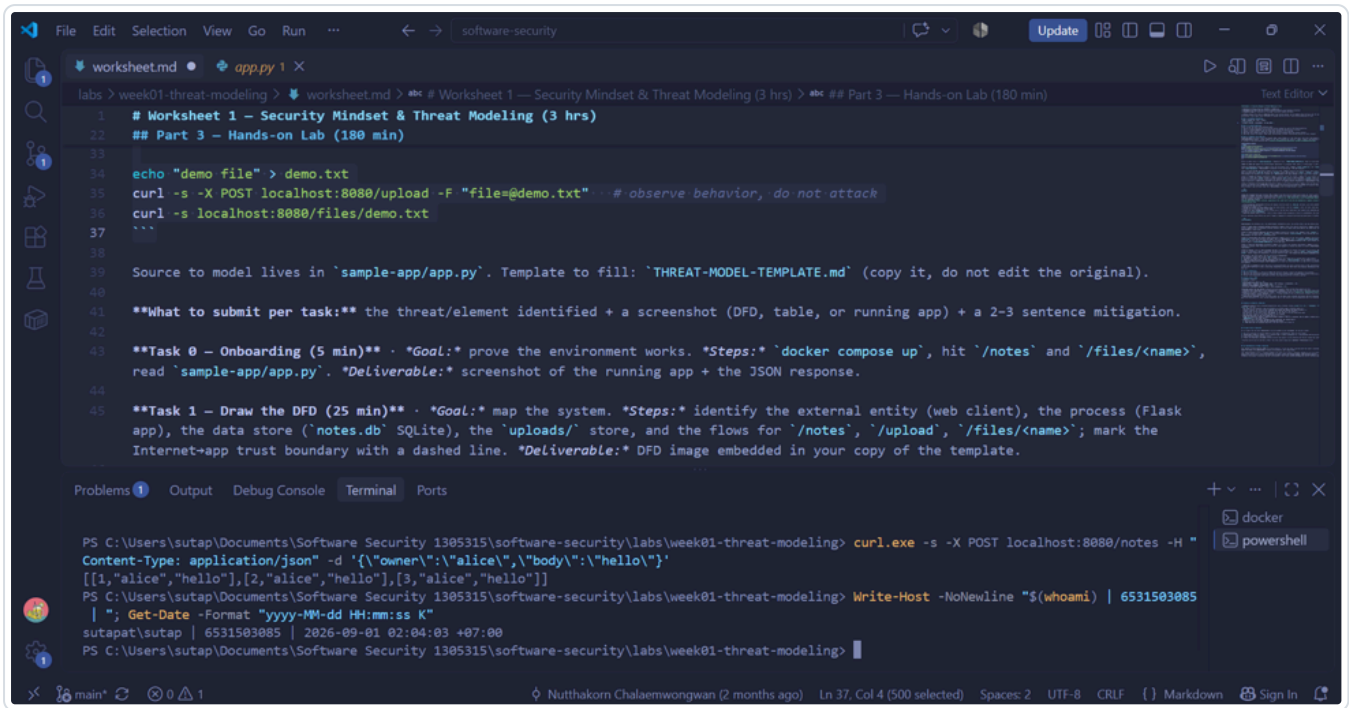
Prerequisites: Docker + Docker Compose in your VM; a drawing tool (draw.io / paper + photo); the Elevation of Privilege deck (print or virtual) — free print-and-play PDF at github.com/adamshostack/eop.

Environment setup

```
cd labs/week01-threat-modeling
docker compose up --build          # starts sample-app on http://localhost:8080
curl -s -X POST localhost:8080/notes -H 'Content-Type: application/json' \
  -d '{"owner":"alice","body":"hello"}' # observe behavior, do not attack
curl -s localhost:8080/notes

echo "demo file" > demo.txt
curl -s -X POST localhost:8080/upload -F "file=@demo.txt" # observe behavior, do not attack
curl -s localhost:8080/files/demo.txt
```

Source to model lives in `sample-app/app.py`. Template to fill: `THREAT-MODEL-TEMPLATE.md` (copy it, do not edit the original).



```
File Edit Selection View Go Run ... software-security Update
labs > week01-threat-modeling > worksheet.md # Worksheet 1 — Security Mindset & Threat Modeling (3 hrs) # Part 3 — Hands-on Lab (180 min)
# Worksheet 1 — Security Mindset & Threat Modeling (3 hrs)
## Part 3 — Hands-on Lab (180 min)

echo "demo file" > demo.txt
curl -s -X POST localhost:8080/upload -F "file=@demo.txt" ...# observe behavior, do not attack
curl -s localhost:8080/files/demo.txt

Source to model lives in `sample-app/app.py`. Template to fill: `THREAT-MODEL-TEMPLATE.md` (copy it, do not edit the original).

**What to submit per task:** the threat/element identified + a screenshot (DFD, table, or running app) + a 2-3 sentence mitigation.

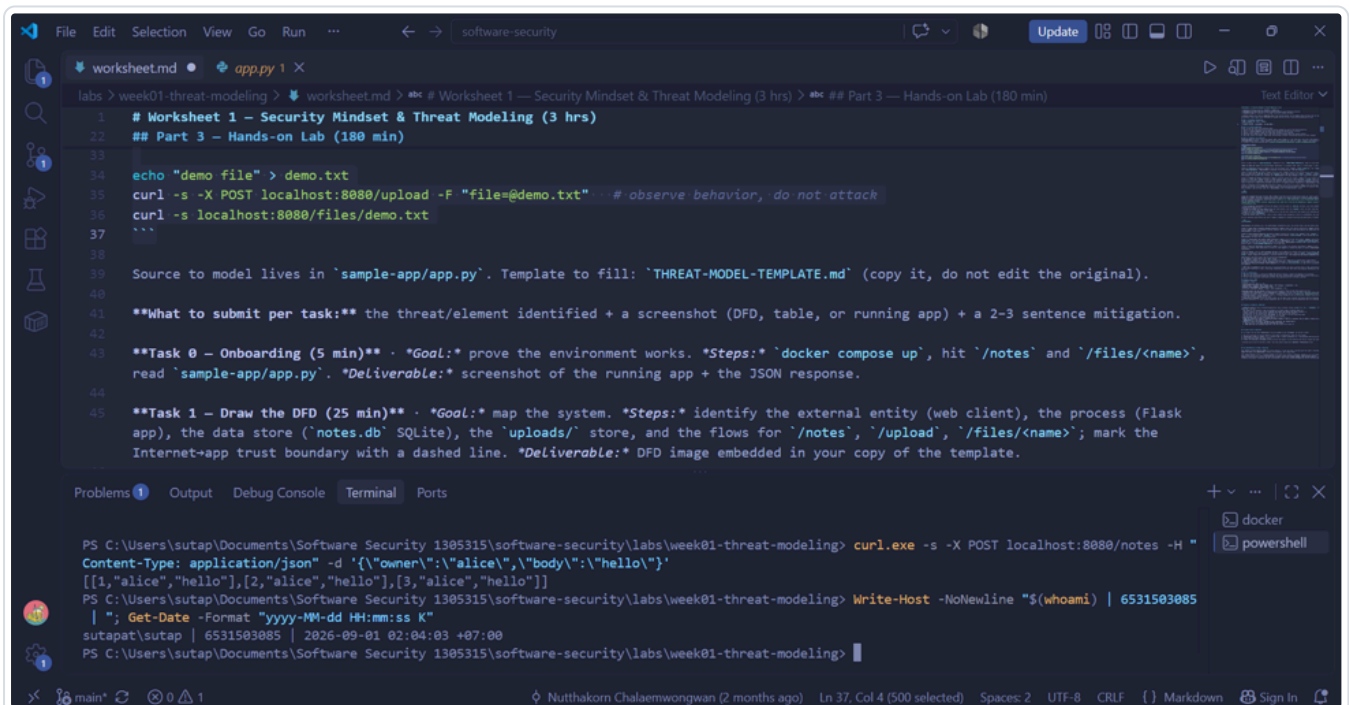
**Task 0 — Onboarding (5 min)** · *Goal:* prove the environment works. *Steps:* `docker compose up`, hit `/notes` and `/files/<name>`, read `sample-app/app.py`. *Deliverable:* screenshot of the running app + the JSON response.

**Task 1 — Draw the DFD (25 min)** · *Goal:* map the system. *Steps:* identify the external entity (web client), the process (Flask app), the data store (`notes.db` SQLite), the `uploads/` store, and the flows for `/notes`, `/upload`, `/files/<name>`; mark the Internet-app trust boundary with a dashed line. *Deliverable:* DFD image embedded in your copy of the template.

Problems Output Debug Console Terminal Ports
PS C:\Users\sutap\Documents\Software Security 1385315\software-security\labs\week01-threat-modeling> curl.exe -s -X POST localhost:8080/notes -H "Content-Type: application/json" -d '{"owner":"alice","body":"hello"}'
[[{"id":1,"owner":"alice","body":"hello"},{"id":2,"owner":"alice","body":"hello"},{"id":3,"owner":"alice","body":"hello"}]]
PS C:\Users\sutap\Documents\Software Security 1385315\software-security\labs\week01-threat-modeling> Write-Host -NoNewline "$(whoami) | 6531503085 | "; Get-Date -Format "yyyy-MM-dd HH:mm:ss K"
sutapat\sutap | 6531503085 | 2026-09-01 02:04:03 +07:00
PS C:\Users\sutap\Documents\Software Security 1385315\software-security\labs\week01-threat-modeling>
```

What to submit per task: the threat/element identified + a screenshot (DFD, table, or running app) + a 2-3 sentence mitigation.

Task 0 — Onboarding (5 min) · *Goal:* prove the environment works. *Steps:* `docker compose up`, hit `/notes` and `/files/<name>`, read `sample-app/app.py`. *Deliverable:* screenshot of the running app + the JSON response.



```
File Edit Selection View Go Run ... software-security Update
labs > week01-threat-modeling > worksheet.md # Worksheet 1 — Security Mindset & Threat Modeling (3 hrs) # Part 3 — Hands-on Lab (180 min)
# Worksheet 1 — Security Mindset & Threat Modeling (3 hrs)
## Part 3 — Hands-on Lab (180 min)

echo "demo file" > demo.txt
curl -s -X POST localhost:8080/upload -F "file=@demo.txt" ...# observe behavior, do not attack
curl -s localhost:8080/files/demo.txt

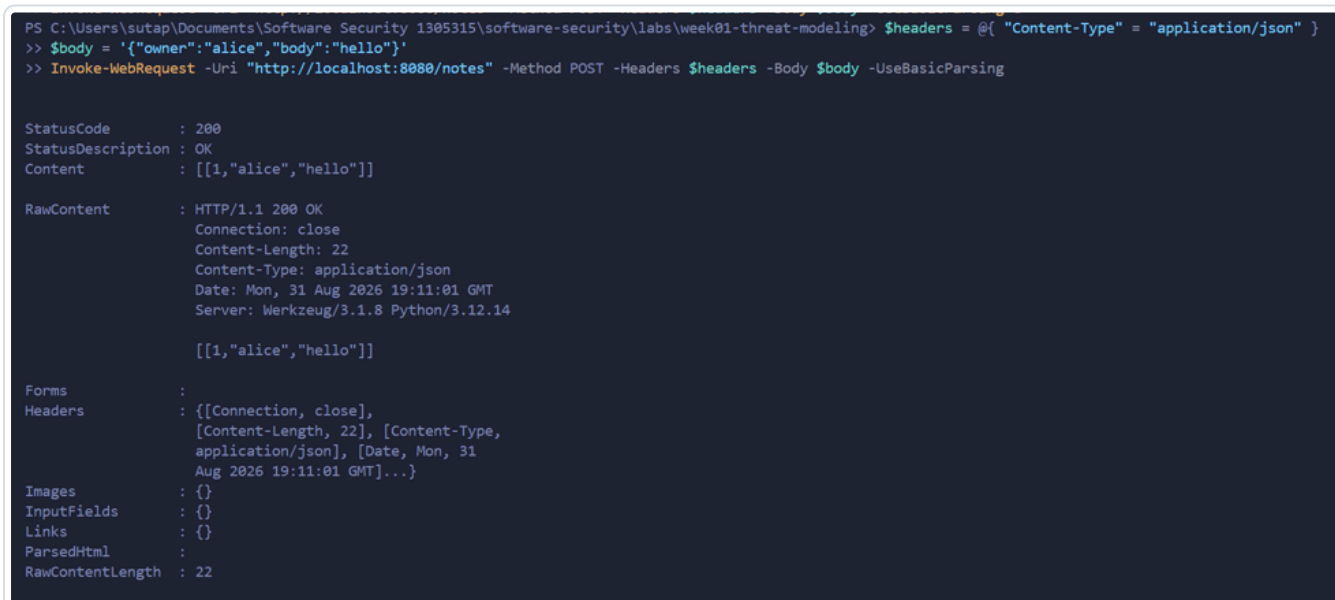
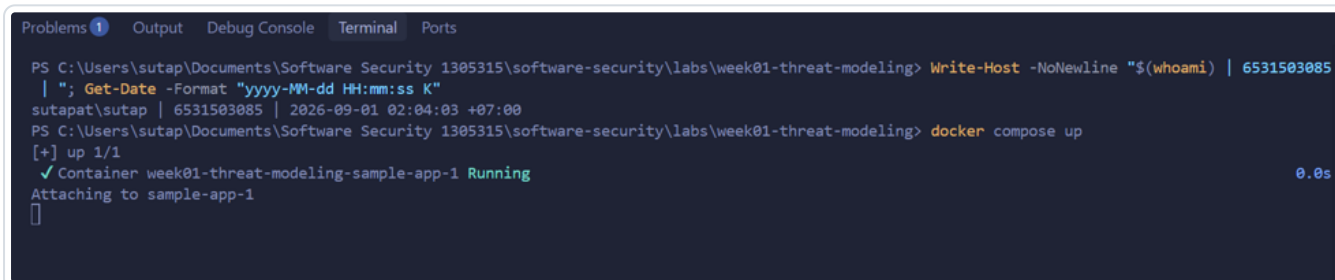
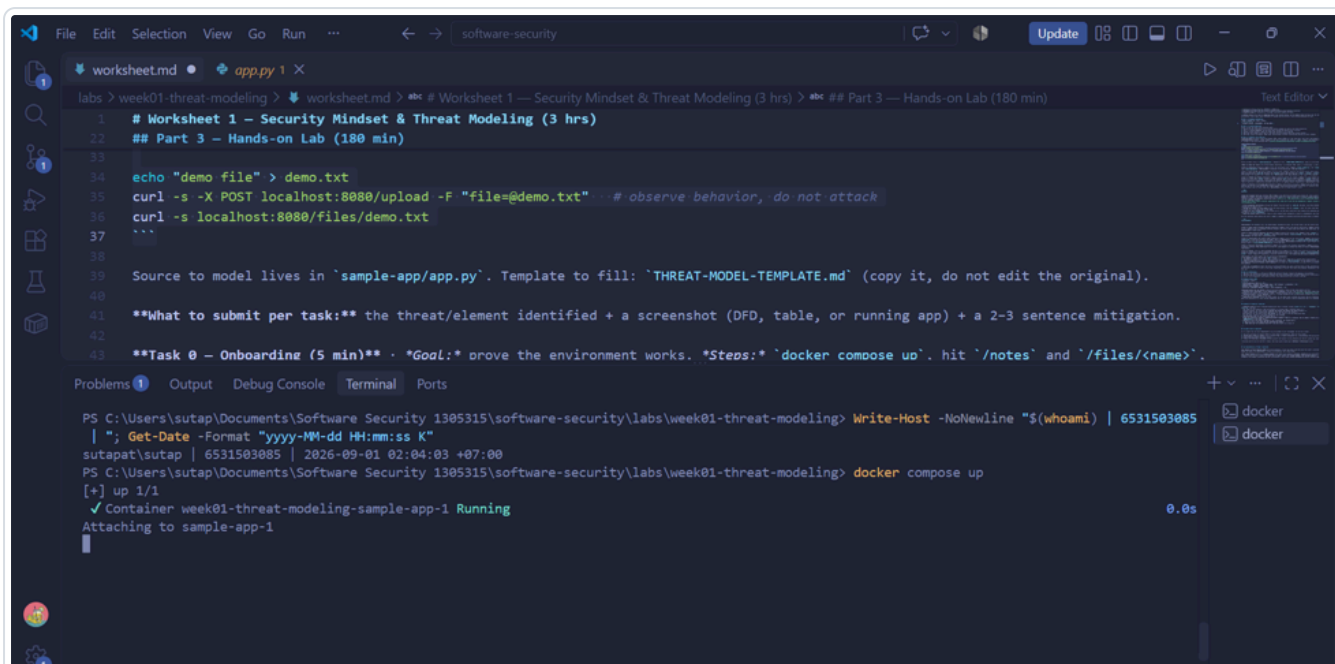
Source to model lives in `sample-app/app.py`. Template to fill: `THREAT-MODEL-TEMPLATE.md` (copy it, do not edit the original).

**What to submit per task:** the threat/element identified + a screenshot (DFD, table, or running app) + a 2-3 sentence mitigation.

**Task 0 — Onboarding (5 min)** · *Goal:* prove the environment works. *Steps:* `docker compose up`, hit `/notes` and `/files/<name>`, read `sample-app/app.py`. *Deliverable:* screenshot of the running app + the JSON response.

**Task 1 — Draw the DFD (25 min)** · *Goal:* map the system. *Steps:* identify the external entity (web client), the process (Flask app), the data store (`notes.db` SQLite), the `uploads/` store, and the flows for `/notes`, `/upload`, `/files/<name>`; mark the Internet-app trust boundary with a dashed line. *Deliverable:* DFD image embedded in your copy of the template.

Problems Output Debug Console Terminal Ports
PS C:\Users\sutap\Documents\Software Security 1385315\software-security\labs\week01-threat-modeling> curl.exe -s -X POST localhost:8080/notes -H "Content-Type: application/json" -d '{"owner":"alice","body":"hello"}'
[[{"id":1,"owner":"alice","body":"hello"},{"id":2,"owner":"alice","body":"hello"},{"id":3,"owner":"alice","body":"hello"}]]
PS C:\Users\sutap\Documents\Software Security 1385315\software-security\labs\week01-threat-modeling> Write-Host -NoNewline "$(whoami) | 6531503085 | "; Get-Date -Format "yyyy-MM-dd HH:mm:ss K"
sutapat\sutap | 6531503085 | 2026-09-01 02:04:03 +07:00
PS C:\Users\sutap\Documents\Software Security 1385315\software-security\labs\week01-threat-modeling>
```



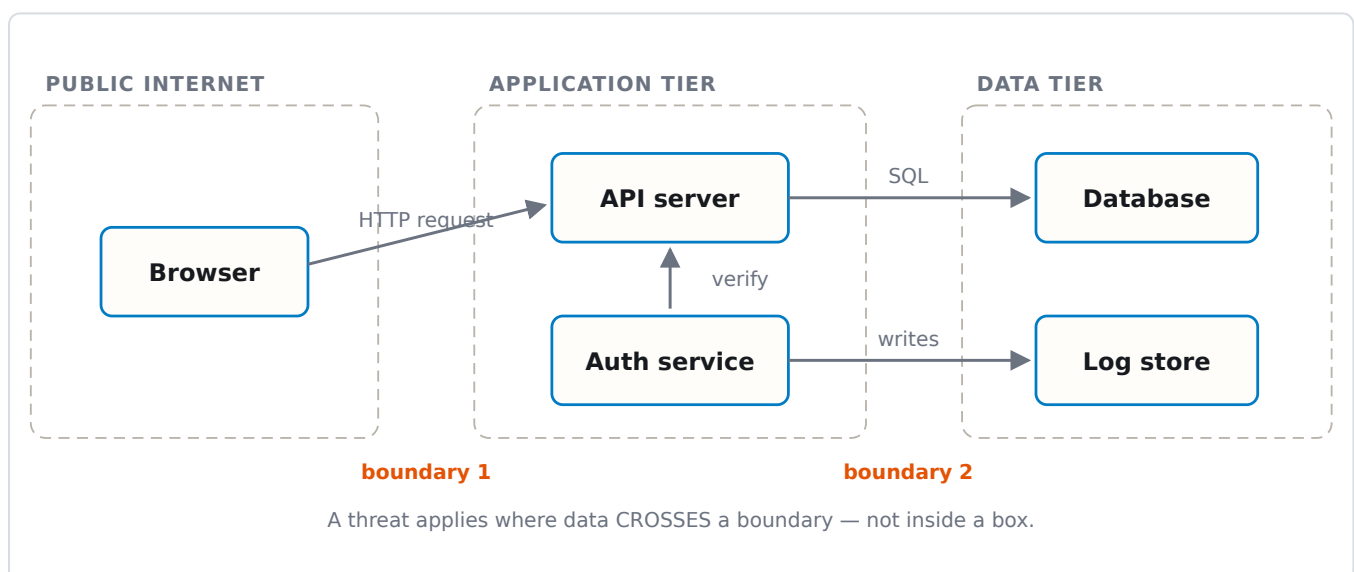
Task 1 — Draw the DFD (25 min) · Goal: map the system. Steps: identify the external entity (web client), the process (Flask app), the data store (`notes.db` SQLite), the `uploads/` store, and the flows for `/notes` , `/upload` , `/files/<name>` ; mark the Internet→app trust boundary with a dashed line. Deliverable: DFD image embedded in your copy of the template.

Task 2 — STRIDE the elements (30 min) · Goal: enumerate threats per element. Steps: for each element fill the S/T/R/I/D/E grid. Ground it in real code: `/notes` accepts a client-supplied `owner` with no auth (Spoofing); `/upload` saves raw `f.filename` — arbitrary-file-write (Tampering) — and echoes the resolved save path back in its response (Information disclosure); `/files/<name>` reads it back but is comparatively defended (see Task 5); no logging anywhere (Repudiation). Deliverable: completed STRIDE table.

Task 3 — Elevation of Privilege game (20 min) · Goal: find threats you missed. Steps: play the EoP deck against your DFD; each card you can tie to a real element/flow scores a point; record every valid threat. No printer or scissors? Draw from the digital deck below instead — same 78 cards, same rule. Deliverable: list of carded threats + score.

eop-deck

Task 3b — Systems-level pass (25 min) 🕒 · Goal: find what the per-element grid cannot see. Tasks 2 and 3 enumerate threats **one element at a time**, and that is exactly where threat models are known to stop short — students taught STRIDE alone reliably identify component threats and *discount system-level ones* (Joshi et al., ASEE 2024). So do a second pass over the **whole** diagram:



- **Trust boundaries end-to-end.** Follow one request from the client to `notes.db` and back. List every boundary it crosses. Which crossing has no check on it?

Ans:

A request from the client to `notes.db` crosses 2 boundaries:

1. Internet → Flask app (Public Internet → Application Tier) — no authentication, no input validation
2. Flask app → `notes.db` / `uploads/` (Application Tier → Data Tier) — no access control, client

data inserted directly

Neither crossing has any check on it.

- **Assume one element is fully owned.** Pick the Flask process, then the `uploads/` store. For each: what does the attacker now *reach* — not what is it, but where does it get them?
Ans:
 1. Flask app owned → attacker can read/modify/delete all notes in `notes.db`, read/overwrite all files in `uploads/`, and path-traverse to the host filesystem via unsanitized filenames.
 2. `uploads/` owned → attacker places malicious files (malware, phishing pages) and any user can download them via `/files/<name>` without authentication — the server becomes a malware distribution point.
- **Chain two "low" findings.** Find two threats you or the EoP deck rated minor that combine into something you would not accept. Write the chain as `A → B → consequence`.
Ans:
`Path traversal via /upload (no secure_filename) → /files/<name> serves any file without auth` → attacker overwrites application files and serves malicious content to all users.
- **One-line system claim.** Finish: "Even if every element-level mitigation in Task 8 is implemented, this system still fails if there is no authentication that any anonymous user on the network can still read, create, and manipulate all data.."

Use the simulation below before you start — toggle a component to attacker-controlled and watch what it reaches:

trust-boundary

Deliverable: the boundary list, two owned-element reachability notes, one written chain, and the system claim.

Task 4 — Abuse cases & attacker personas (20 min) · *Goal:* think like specific adversaries. *Steps:* define 2 personas (e.g. a curious logged-in user; an anonymous internet attacker) and write 2 abuse cases each against the sample app, tied to DFD elements. *Deliverable:* 4 abuse cases.

Ans:

Persona 1: Curious student that have a basic programming, knows how to use curl or Postman. They want to snoop on other people's data for fun

Abuse Case 1: The student sends a simple GET request to `/notes` and sees all notes from every user because there is no access control. They can read everyone's private notes without logging in.

Tied to: `/notes GET` → Information Disclosure

Abuse Case 2: The student sends a POST to `/notes` with "owner": "professor" to create a fake note pretending to be the professor. There is no authentication so the system accepts it.

Tied to: `/notes POST` → Spoofing

Persona 2: Anonymous attacker they know about common web vulnerabilities like path traversal and want to steal data or damage the system without physical access.

Abuse Case 3: The attacker sends rapidly POST/upload large files to disk. Until it full up the disk.

Tied to: /upload POST → Denial of Service

Abuse Case 4: The attacker uploads a file with the name ../../app.py via POST /upload. Because the app uses f.filename directly without sanitizing it the file gets saved outside the uploads/ folder and could overwrite important system files.

Tied to: /upload POST → Tampering

Task 5 — Path-traversal deep-dive (25 min) · Goal: analyze the riskiest flow. Steps: trace /upload → /files/<name>; explain how ../../ in a filename escapes uploads/; sketch the secure design (secure_filename, store outside web root, allow-list extensions). Deliverable: the data flow + secure-design note.

Ans: Path-traversal deep-dive

Data flow

Client sends POST /upload with a file → Flask takes f.filename directly and saves it to uploads/ folder using os.path.join(UPLOAD_DIR, f.filename) → Client sends GET /files/ → Flask returns the file from uploads/ using send_from_directory().

How path traversal works

The app uses f.filename directly without sanitizing it. If an attacker sends a filename like ../../app.py, the ../../ means "go up one directory." So os.path.join("uploads", "../../app.py") saves the file outside the uploads/ folder. The attacker could overwrite system files like the app source code or configuration files.

Secure design note

1. Use secure_filename() from werkzeug.utils to strip ../../ from filenames so path traversal is not possible

2. Allow-list file extensions — only accept safe types like .txt, .pdf, .png and reject everything else

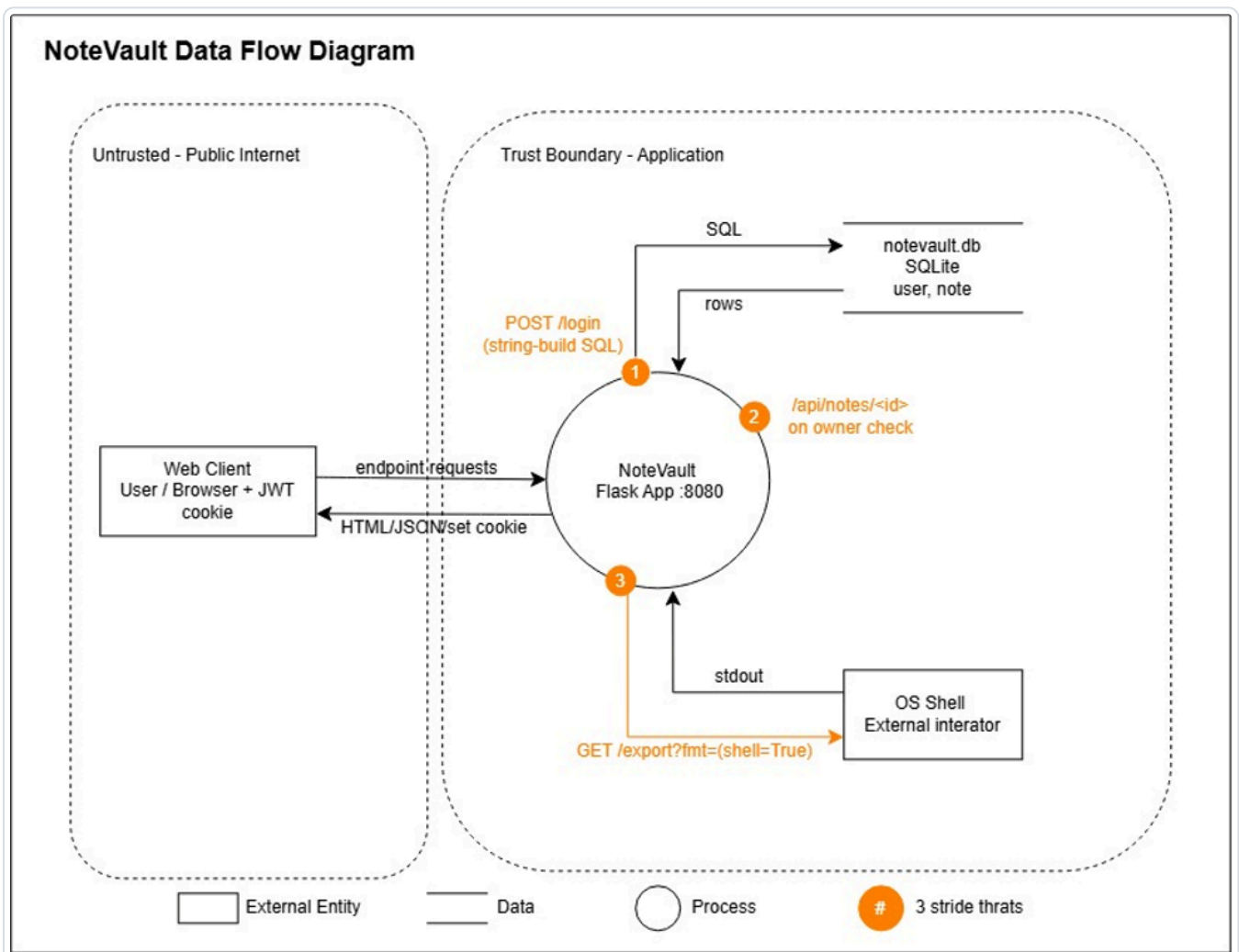
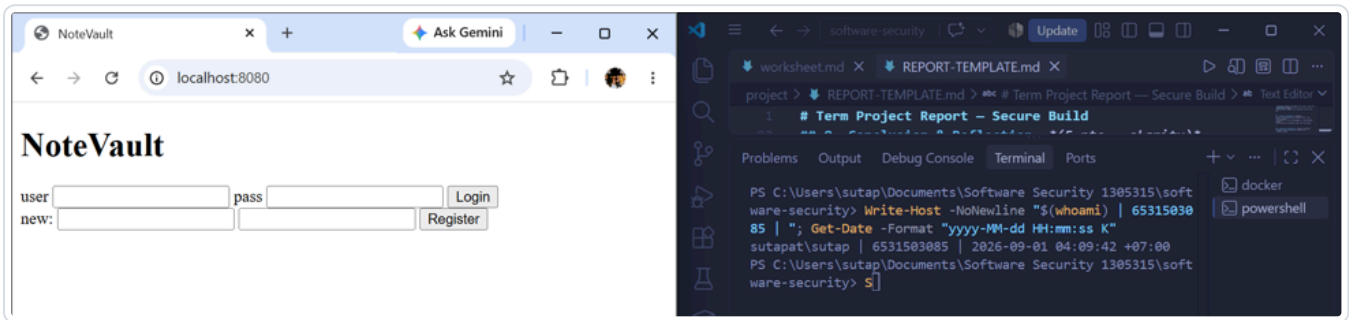
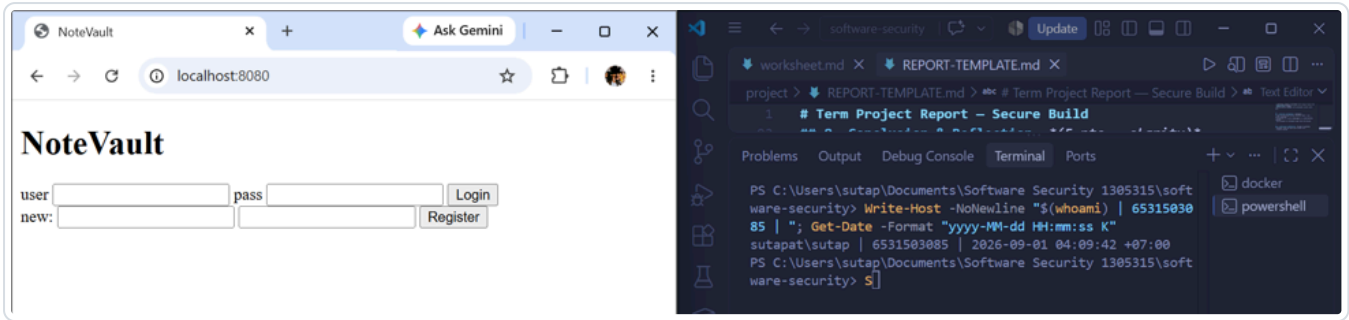
3. Add authentication — require login before uploading so anonymous users cannot upload files.

4. Store files outside web root, so, move uploads/ to a location not directly accessible by the web server (e.g. /var/data/uploads/)

5. Class fix principle: no user-supplied string should ever be used directly as a path component. This prevents the entire category of path traversal, not just one endpoint.

Task 6 — Threat-model the project target (30 min) · Goal: kick off your term project.

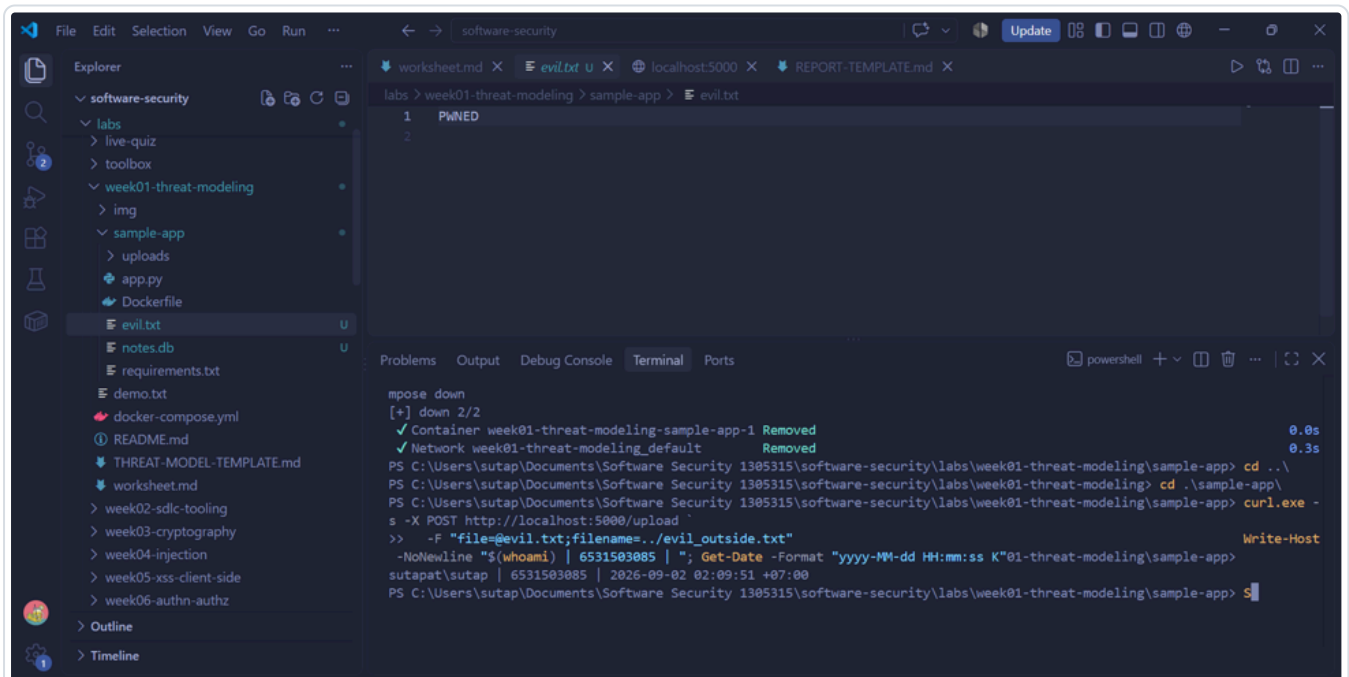
Steps: stop the sample-app first (docker compose down — both apps bind host port 8080), then run NoteVault (cd ../../project/starter-app && docker compose up), draw a quick DFD, and list the top 3 STRIDE threats you'd investigate. Deliverable: NoteVault DFD + top-3 threats (reuse these in your project report — project/REPORT-TEMPLATE.md in the repo root).

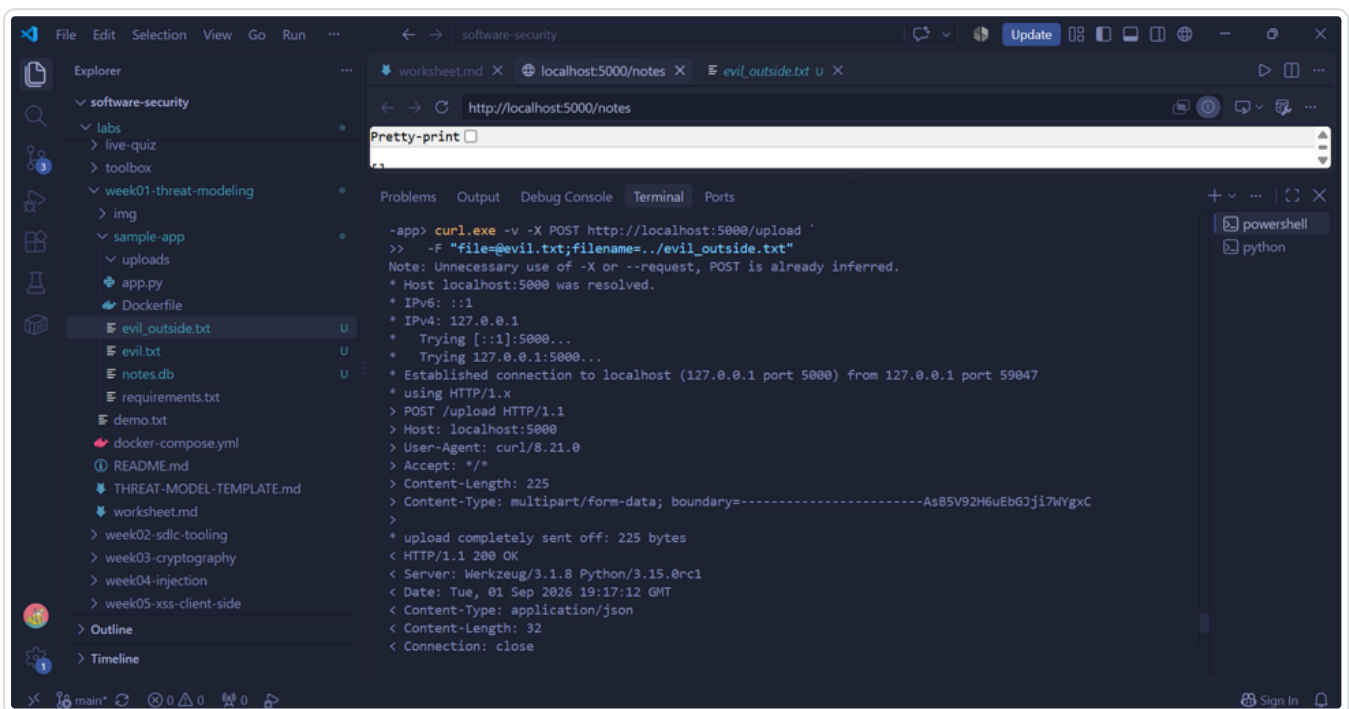
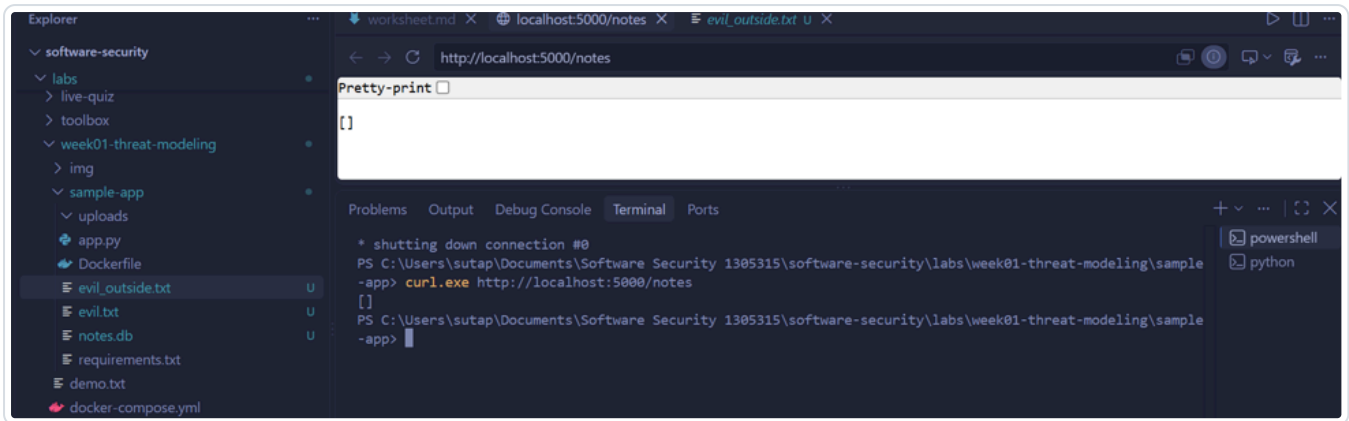


Top 3 STRIDE threats of NoteVault
Endpoint | STRIDE | Threats

- Task 7 — Security requirements (15 min)** · *Goal:* turn threats into testable requirements. *Steps:* write 3 security requirements as acceptance criteria ("the system must ... so that ..."), each mapped to a threat from Task 2 or Task 6. *Deliverable:* 3 testable security requirements.
- Ans:

- Task 8 — Defend / fix it: rank & mitigate (25 min)** 🛡️ · Goal: turn threats into action you can prove. Steps: rank the top 5 threats by likelihood × impact; propose one concrete mitigation each (e.g., auth on `/notes`, `secure_filename()` + allowlist for `/upload`, request logging for Repudiation, size/rate limits for DoS). Then **pick one and actually implement it** in your fork.





It is clear that the server accepts filenames with .. / and responded with a 200 OK without any checks at all. Keep this block as evidence "before."

Add from werkzeug.utils import secure_filename (new import)

UPLOAD_DIR changes to os.path.abspath("uploads") (to ensure accurate path comparison) + add ALLOWED_EXTENSIONS

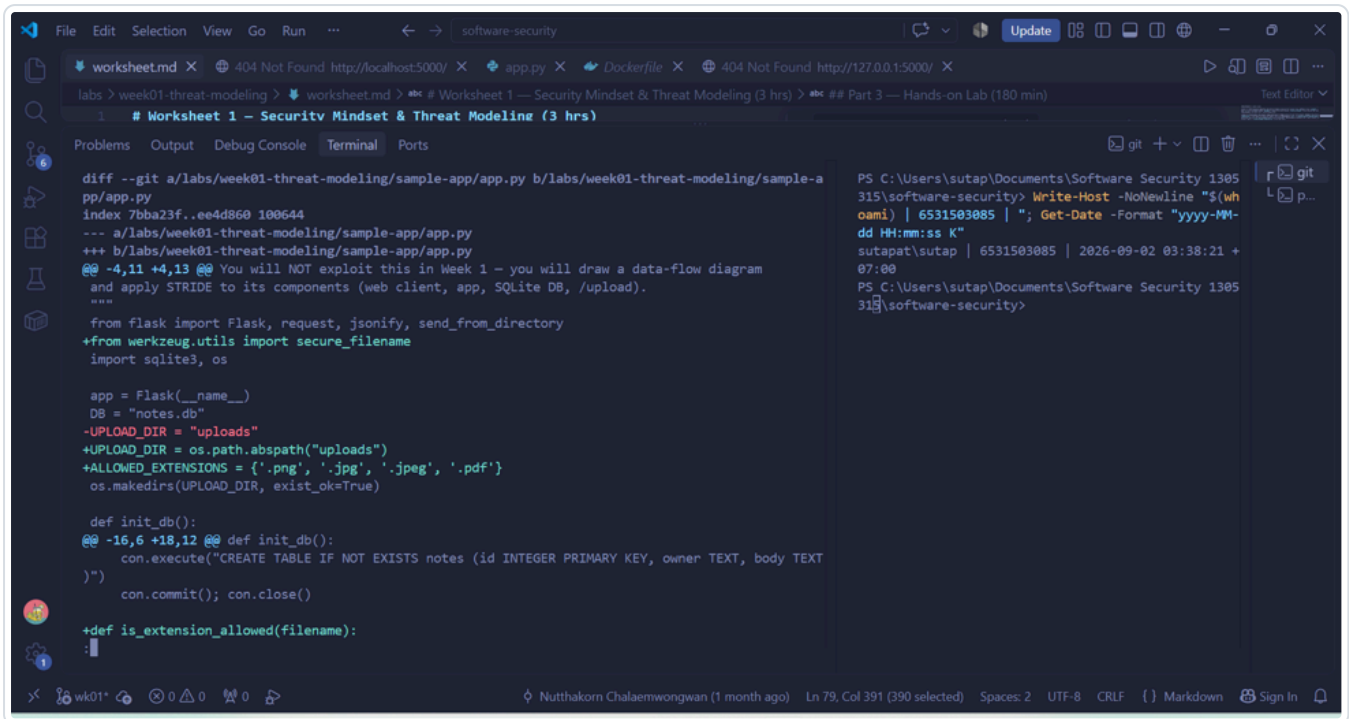
Add the function is_extension_allowed()

The upload() function has been completely rewritten (as we agreed).

/notes, /files/, init_db() remain unchanged, untouched.

Deliverable — the top-5 table, plus for the one you implemented:

1. the **diff** (commit hash on your **wk01** branch),



2. **evidence it works:** the request that succeeded before your change and is refused after — both outputs,
/*

Evidence — Before fix

```
curl.exe -v -X POST http://localhost:5000/upload -F "file=@evil.txt;filename=../evil_outside.txt"
...
< HTTP/1.1 200 OK
{"saved":"../evil_outside.txt"}
```

File `evil_outside.txt` appeared outside `uploads/`, confirming path traversal.

Evidence — After fix

```
curl.exe -v -X POST http://localhost:5000/upload -F "file=@evil.txt;filename=../evil_outside2.txt"
...
< HTTP/1.1 415 UNSUPPORTED MEDIA TYPE
{"error":"Extension not allowed"}
```

Same request now rejected.

*/

3. **why it closes the class, not the instance** (2–3 sentences). `secure_filename()` on one endpoint is an instance fix; "no user-supplied string ever becomes a path component" is a class fix. Say which yours is, and if it's an instance fix, say what the class fix would be.

Ans:

The allow-list, `secure_filename()`, and path-verification logic are only used in the `/upload` route handler. This means that if later on an endpoint is added that saves a file with a name given by the client, the arbitrary-file-write vulnerability could show up again. A real class fix would make sure that "no user-supplied string ever becomes a path component" across the whole system. This could be done by always creating the stored filename on the server (for example, using a UUID) and only keeping the client's original filename as metadata, or by making sure that every endpoint calls the same, checked `safe_save()` helper for every file save.

Why this is weighted. Fewer than half of working developers can spot a security hole in code, and being shown vulnerabilities does not by itself teach you to find or close them. Exploiting is the half that feels like progress; defending is the half that transfers to your job.

Part 4 — Reflection

1. Map your top finding to a CWE and to OWASP A06 (Insecure Design); explain the mapping in one sentence.

Ans: This finding relates to CWE-22 (Path Traversal) and OWASP A06 (Insecure Design) because the system worked exactly as it was supposed to, accepting a file path without ever checking it against a trusted directory boundary. This means that the flaw is a missing design control and not an implementation bug that messed up logic that should have worked.

2. Name one real-world breach caused by a design flaw (not a missing patch) and what design control would have prevented it.

Ans: In the 2019 Capital One breach, a former AWS employee used an SSRF flaw in a badly set up WAF to get into the private EC2 metadata service and steal credentials from an IAM role. This let over 100 million customers' data become public. Enforcing IMDSv2, which needs a session token before returning credentials and blocks easy SSRF-based access, would have stopped the breach, even though the WAF still had the same SSRF flaw.

3. Of your five mitigations, which gives the most risk reduction per unit of effort, and why?

Ans: The `/upload` fix (allow-list + `secure_filename()` + path verification) lowers risk the most for the least amount of work. It only required about 15 lines of code and didn't change the architecture, but it closed a vulnerability that could let anyone write to any file, which was the most serious finding in the top-5 table. For relatively less serious risks (DoS, Repudiation), defences like rate-limiting or centralised logging need more ongoing infrastructure (e.g., Flask-Limiter, log storage/monitoring).

Grading rubric (100)

Criterion	Points
Lecture questions (Part 2)	20
Exploitation + evidence (DFD + STRIDE table + EoP findings + screenshots)	40
Defense (top-5 ranking + mitigations)	25
Reflection (CWE/OWASP mapping + breach + best mitigation)	15

Assessed within the rows above (they are not extra points — they are what those points are for):

- **Systems-level reasoning** (inside *Exploitation + evidence*, Task 3b): does the model reach past single elements to boundaries, reachability and chains? Scored with the STRIDE + systems-thinking rubrics of [Joshi et al. 2024](#).
- **Defensive proof** (inside *Defense*, Task 8): a claimed mitigation with no before/after evidence scores at most half. A mitigation you can show closing a *class* scores full.
- **Adversarial thinking** (across the whole sheet): do the abuse cases, personas and chains show you reasoning as an attacker with goals and constraints — or just listing categories? This is the course's central disposition and it is assessed, not assumed.

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- **Personalized flag (if this lab issues one):** _____
Flags are unique per student — submitting another student's flag is a violation. How to submit: learn.zcr.ai/submit (full guide: [SUBMISSION.md](#) in the repo root).
- **Explain in your own words** (*graded on your reasoning, not copied text*):
 1. What did you do, and **why did the vulnerability work?**
 2. **Why does your fix actually stop it** — and what could still break it?



Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.
/* The vulnerability exists because `f.filename` accepts raw input from the client.
An attacker can upload a file named `../.././etc/passwd` or

.././../var/www/html/backdoor.py, allowing them to escape the UPLOAD_DIR and overwrite critical files on the server.

To fix this, use Werkzeug's built-in `secure_filename()` utility, which strips out directory traversal elements (like `../` and `/`) and ensures the filename is safe for the filesystem.

```
import os
from flask import request
from werkzeug.utils import secure_filename

@app.route("/upload", methods=["POST"])
def upload():
    if "file" not in request.files:
        return {"error": "No file uploaded"}, 400
    f = request.files["file"]
    if f.filename == '':
        return {"error": "No file selected"}, 400
    safe_filename = secure_filename(f.filename)
    if not safe_filename:
        return {"error": "Invalid filename"}, 400
    f.save(os.path.join(UPLOAD_DIR, safe_filename))
    return {"saved": safe_filename}
/
```

2. Find *what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).

```
//safe_filename = secure_filename(f.filename)
//...
//f.save(os.path.join(UPLOAD_DIR, safe_filename))
```

3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

```
/*ALLOWED_EXTENSIONS = {'.png', '.jpg', '.jpeg', '.pdf'}

def is_extension_allowed(filename):
    if not filename:
        return False
    _, ext = os.path.splitext(filename)
    return ext.lower() in ALLOWED_EXTENSIONS

@app.route("/upload", methods=["POST"])
def upload():
    if "file" not in request.files:
        return jsonify({"error": "No file part"}), 400
    f = request.files["file"]
    if f.filename == '':
        return jsonify({"error": "No selected file"}), 400
    if not is_extension_allowed(f.filename):
        return jsonify({"error": "Extension not allowed"}), 415
    filename = secure_filename(f.filename)
```



```

if not filename:
    return jsonify({"error": "Invalid filename"}), 400
target_path = os.path.join(UPLOAD_DIR, filename)
real_path = os.path.abspath(target_path)
if not real_path.startswith(UPLOAD_DIR + os.sep):
    return jsonify({"error": "Path traversal attempt detected"}), 403
f.save(real_path)
return jsonify({"message": "File uploaded successfully", "filename": filename}), 200
*/

```

Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

Ans: Ans: The /upload endpoint lets the client set the name of the file that will be saved. The server then takes that value (f.filename) and combines it with the path to the upload folder to find out where to save the file, without testing it first. This lets a bad guy change the name of the file to something like ../evil.txt. Since the operating system sees ../ as a command to move up one directory, the file is saved outside of the intended uploads/ folder instead of inside it. The endpoint can be hacked not because of a single missing check, but because input from the client that can't be trusted can directly change a filesystem path.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**.

Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).

Ans: Ans: Fix this Flask /upload endpoint. It has two separate vulnerabilities: (1) path traversal, and (2) unrestricted file type. Fix both. For the path traversal fix, don't rely on filename sanitization alone — also verify that the final resolved save path is still inside the intended upload directory. For the file type fix, only allow a specific extension allow-list (e.g., .png, .jpg, .pdf).

```

/app.route("/upload", methods=["POST"])
def upload():
    f = request.files["file"]
    f.save(os.path.join(UPLOAD_DIR, f.filename))
    return {"saved": f.filename}
/

```